

Databases – MongoDB, PostgreSQL, MySQL & Redis Complete Guide

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 26 Oct 2026 – 8 Nov 2026 (Days 71–84)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm** & **9:00–10:00pm** · Sat–Sun **9:00am–12:00pm**, **3:00–5:30pm**, **6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import [calendar/mern-mastery-study.ics](https://calendar.google.com/calendar/ical/harsh.gupta%40getreelax.com/public/calendar/mern-mastery-study.ics)

Goal: Master database design for MERN production stacks — MongoDB, PostgreSQL, MySQL, Redis, Mongoose, Prisma, SQL, transactions, indexing, and choosing the right database.

Table of Contents

1. SQL vs NoSQL — When to Use What
2. CAP Theorem & ACID vs BASE
3. MongoDB Deep Dive
4. Mongoose ODM
5. PostgreSQL Deep Dive
6. MySQL Deep Dive
7. Prisma ORM
8. SQL Joins & Queries
9. Transactions
10. Indexing Strategies
11. Aggregation & Analytics
12. Redis Caching Patterns
13. Database Architecture Patterns

14. Interview Q&A

15. Practice Links & Resources

1. SQL vs NoSQL – When to Use What

High-Level Comparison

Factor	PostgreSQL (SQL)	MySQL (SQL)	MongoDB (NoSQL)	Redis (In-Memory)
Data model	Tables, rows, columns	Tables, rows, columns	Documents (BSON)	Key-value, lists, sets
Schema	Fixed, enforced	Fixed, enforced	Flexible, optional	Schema-less
Relations	JOINS, foreign keys	JOINS, foreign keys	Embed or reference	No relations
Transactions	Full ACID	Full ACID (InnoDB)	Multi-doc ACID (v4+)	Limited (MULTI/EXEC)
Query language	SQL	SQL	MongoDB Query API	Commands
Scale	Vertical + read replicas	Vertical + read replicas + Vitess	Horizontal sharding	Cluster, replication
Best for	Finance, GIS, complex queries	High-read web apps, e-commerce	Content, catalogs, logs	Cache, sessions, queues
Used at	Goldman, Stripe, Instagram	Flipkart, Uber, Airbnb, Netflix	SaaS, real-time apps	Everywhere (cache)

Decision Matrix

Requirement	Choose
Complex joins, window functions, JSONB	PostgreSQL
Massive read-heavy web traffic, simple schema	MySQL
Team already on LAMP/MEAN with MySQL	MySQL
AWS RDS default familiarity	PostgreSQL or MySQL (both on RDS)
Flexible/evolving schema	MongoDB
Strong consistency + advanced analytics	PostgreSQL
High write throughput, time-series	MongoDB
Sub-millisecond reads	Redis
Session storage, rate limiting	Redis

Polyglot Persistence (Production Pattern)

PostgreSQL
Analytics
Reporting
Payments

MySQL
Core app
Orders*
Users*

MongoDB
Catalog
Activity
Logs

Redis
Sessions
Cache
Rate limit

* Many teams pick ONE SQL DB: PostgreSQL OR MySQL – not always both

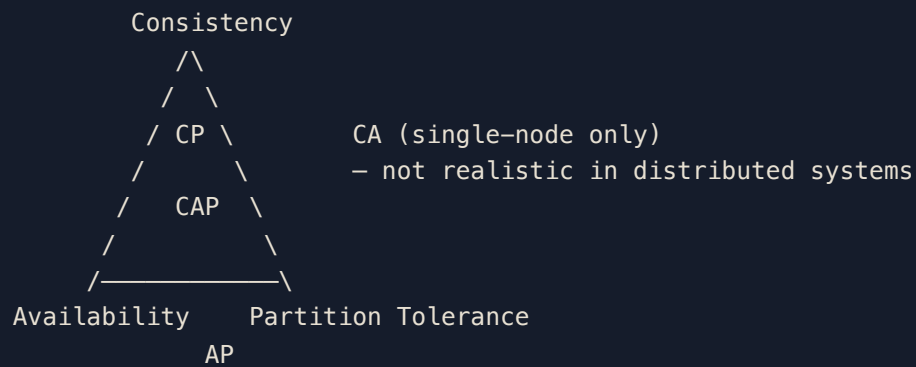
Interview answer: "Use **PostgreSQL** for complex queries, JSONB, and strict ACID reporting. Use **MySQL** for high-read OLTP web apps — very common at Flipkart, Uber, and Indian product companies. **MongoDB** for flexible schemas. **Redis** for caching and sessions. Many MERN stacks use MySQL or PostgreSQL as primary SQL store plus MongoDB for specific workloads plus Redis for cache."

2. CAP Theorem & ACID vs BASE

CAP Theorem

In a distributed system, you can guarantee at most **two of three**:

Property	Meaning
C onsistency	Every read returns the latest write
A vailability	Every request gets a response
P artition tolerance	System works despite network failures



System	CAP Choice	Example
PostgreSQL (single node)	CA	Traditional RDBMS
MongoDB (replica set)	CP	Consistency over availability during partition
Cassandra	AP	Availability + partition tolerance
Redis (cluster)	AP	Eventual consistency in cluster mode

ACID vs BASE

Property	ACID (SQL)	BASE (NoSQL)
A tomicity	All or nothing	Eventual
C onsistency	Immediate	Soft state
I solation	Serializable	–
D urability	Guaranteed	Basically available
Trade-off	Strong guarantees	Scale + availability

3. MongoDB Deep Dive

Document Model

MongoDB stores data as **BSON documents** (binary JSON) in **collections**.

```
// users collection
{
  _id: ObjectId("507f1f77bcf86cd799439011"),
  name: "Amit Kumar",
  email: "amit@example.com",
  role: "admin",
  profile: { // embedded document
    bio: "Full-stack developer",
    avatar: "https://cdn.example.com/a.jpg",
    social: {
      twitter: "@amit",
      github: "amitdev"
    }
  },
  tags: ["javascript", "nodejs"], // array
  address: {
    city: "Bangalore",
    country: "IN",
    coordinates: [77.5946, 12.9716] // [longitude, latitude]
  },
  createdAt: ISODate("2024-01-15T10:00:00Z"),
  updatedAt: ISODate("2024-06-01T08:30:00Z")
}
```

Embedding vs Referencing

Pattern	When to Use	Example
Embed	One-to-few, data accessed together	User → address, order → line items
Reference	One-to-many, many-to-many, large arrays	User → orders, post → comments
Hybrid	Frequently accessed subset	User embeds last 5 orders, references rest

```
// ❌ Unbounded array – document grows forever
{
  userId: ObjectId("..."),
  comments: [ /* millions of comments */ ]
}

// ✅ Reference pattern
// comments collection
{ _id: ObjectId("..."), postId: ObjectId("..."), text: "Great post!", userId: Obje
```

CRUD Operations

```
// Create
db.users.insertOne({ name: "Amit", email: "a@x.com", role: "user" });
db.users.insertMany([ { name: "Bob" }, { name: "Carol" } ]);

// Read
db.users.findOne({ email: "a@x.com" });
db.users.find({ role: "admin" }).sort({ createdAt: -1 }).limit(10).skip(20);
db.users.find({ "address.city": "Bangalore" }); // dot notation
db.users.find({ tags: { $in: ["nodejs", "react"] } }); // array query
db.users.find({ age: { $gte: 18, $lte: 65 } }); // range

// Update
db.users.updateOne(
  { email: "a@x.com" },
  { $set: { name: "Amit K" }, $inc: { loginCount: 1 }, $push: { tags: "mongodb" } }
);

// Delete
db.users.deleteOne({ email: "a@x.com" });
db.users.deleteMany({ role: "guest", lastLogin: { $lt: new Date("2023-01-01") } })
```

Query Operators Reference

Operator	Purpose	Example
<code>\$eq</code> , <code>\$ne</code>	Equal / not equal	<code>{ status: { \$ne: "deleted" } }</code>
<code>\$gt</code> , <code>\$gte</code> , <code>\$lt</code> , <code>\$lte</code>	Comparison	<code>{ price: { \$gte: 100 } }</code>
<code>\$in</code> , <code>\$nin</code>	In / not in array	<code>{ role: { \$in: ["admin", "mod"] } }</code>
<code>\$and</code> , <code>\$or</code> , <code>\$nor</code>	Logical	<code>{ \$or: [{ status: "active" }, { premium: true }] }</code>
<code>\$exists</code>	Field exists	<code>{ deletedAt: { \$exists: false } }</code>
<code>\$regex</code>	Pattern match	<code>{ name: { \$regex: "^Am", \$options: "i" } }</code>
<code>\$elemMatch</code>	Array element match	<code>{ scores: { \$elemMatch: { \$gte: 80 } } }</code>

4. Mongoose ODM

Mongoose provides **schema validation**, **middleware**, **population**, and **TypeScript support** for MongoDB.

Schema Definition

```
import { Schema, model, Document, Types } from "mongoose";

interface IUser extends Document {
  name: string;
  email: string;
  passwordHash: string;
  role: "user" | "admin";
  profile: {
    bio?: string;
    avatar?: string;
  };
  tags: string[];
  createdAt: Date;
  updatedAt: Date;
}

const userSchema = new Schema<IUser>(
  {
    name: { type: String, required: [true, "Name is required"], trim: true, maxlength: 50 },
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^\S+@\S+\.\S+$/, "Invalid email format"],
    },
    passwordHash: { type: String, required: true, select: false },
    role: { type: String, enum: ["user", "admin"], default: "user" },
    profile: {
      bio: { type: String, maxlength: 500 },
      avatar: String,
    },
    tags: [{ type: String, trim: true }],
  },
  {
    timestamps: true,
    toJSON: {
      transform(_doc, ret) {
        delete ret.passwordHash;
        delete ret.__v;
        return ret;
      },
    },
  }
);

// Indexes
```

```
userSchema.index({ email: 1 });
userSchema.index({ name: "text", "profile.bio": "text" }); // text search
userSchema.index({ role: 1, createdAt: -1 }); // compound

export const User = model<IUser>("User", userSchema);
```

Middleware (Hooks)

```
import bcrypt from "bcrypt";

// Pre-save - hash password before saving
userSchema.pre("save", async function (next) {
  if (!this.isModified("passwordHash")) return next();
  this.passwordHash = await bcrypt.hash(this.passwordHash, 12);
  next();
});

// Post-save - log creation
userSchema.post("save", function (doc) {
  console.log(`User created: ${doc.email}`);
});
```

Population (Joins)

```
const postSchema = new Schema({
  title: String,
  author: { type: Types.ObjectId, ref: "User", required: true },
  comments: [{ type: Types.ObjectId, ref: "Comment" }],
});

// Populate author when fetching posts
const posts = await Post.find()
  .populate("author", "name email avatar")
  .populate({ path: "comments", options: { limit: 5, sort: { createdAt: -1 } } })
  .lean(); // returns plain JS objects - faster, no Mongoose overhead
```

Mongoose CRUD with TypeScript

```
// Create
const user = await User.create({ name: "Amit", email: "a@x.com", passwordHash: "pl

// Read
const users = await User.find({ role: "admin" })
  .select("name email")
  .sort({ createdAt: -1 })
  .limit(10)
  .lean();

// Update
const updated = await User.findByIdAndUpdate(
  id,
  { $set: { name: "New Name" } },
  { new: true, runValidators: true }
);

// Delete
await User.findByIdAndDelete(id);

// Upsert
await User.findOneAndUpdate(
  { email: "a@x.com" },
  { $set: { lastLogin: new Date() } },
  { upsert: true, new: true }
);
```

5. PostgreSQL Deep Dive

Normalized Schema Design

```
-- Users table
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(100) NOT NULL,
  email       VARCHAR(255) NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  role        VARCHAR(20) NOT NULL DEFAULT 'user' CHECK (role IN ('user', 'admin'))
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Orders table
CREATE TABLE orders (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  status      VARCHAR(20) NOT NULL DEFAULT 'pending'
              CHECK (status IN ('pending', 'paid', 'shipped', 'delivered', 'cancel
  total      DECIMAL(10, 2) NOT NULL CHECK (total >= 0),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Order items (line items)
CREATE TABLE order_items (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id    UUID NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
  product_id  UUID NOT NULL REFERENCES products(id),
  quantity    INTEGER NOT NULL CHECK (quantity > 0),
  unit_price  DECIMAL(10, 2) NOT NULL,
  UNIQUE (order_id, product_id)
);

-- Products
CREATE TABLE products (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(200) NOT NULL,
  slug        VARCHAR(200) NOT NULL UNIQUE,
  price       DECIMAL(10, 2) NOT NULL,
  stock       INTEGER NOT NULL DEFAULT 0,
  category_id UUID REFERENCES categories(id),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Indexes
CREATE INDEX idx_orders_user_id ON orders(user_id);
```

```
CREATE INDEX idx_orders_status_created ON orders(status, created_at DESC);
CREATE INDEX idx_order_items_order_id ON order_items(order_id);
CREATE INDEX idx_products_category ON products(category_id);
CREATE INDEX idx_products_slug ON products(slug);
```

Normalization Levels

Form	Rule	Example
1NF	Atomic values, no repeating groups	Separate order_items from orders
2NF	No partial dependencies	Product name in products, not order_items
3NF	No transitive dependencies	Category name in categories, not products
Denormalize	When read performance critical	Cache <code>order_count</code> on users table

Data Types Reference

Type	Use Case	Example
<code>UUID</code>	Primary keys	<code>gen_random_uuid()</code>
<code>VARCHAR(n)</code>	Variable strings	Names, emails
<code>TEXT</code>	Long strings	Descriptions, content
<code>INTEGER / BIGINT</code>	Whole numbers	Counts, IDs
<code>DECIMAL(p,s)</code>	Money, precise numbers	<code>DECIMAL(10,2)</code>
<code>BOOLEAN</code>	True/false	<code>is_active</code>
<code>TIMESTAMPZ</code>	Dates with timezone	Always use <code>TIMESTAMPZ</code>
<code>JSONB</code>	Semi-structured data	Metadata, settings
<code>ARRAY</code>	Lists	Tags, permissions
<code>ENUM</code>	Fixed set (or use <code>CHECK</code>)	Status values

6. MySQL Deep Dive

MySQL is the world's most deployed open-source relational database. **InnoDB** is the default storage engine (ACID, row-level locking, foreign keys). Very common at **Flipkart**, **Uber**, **Airbnb**, **Netflix**, and many Indian product companies — often the SQL database you'll see in interviews alongside PostgreSQL.

PostgreSQL vs MySQL – Interview Comparison

Factor	PostgreSQL	MySQL (InnoDB)
Default isolation	Read Committed	Repeatable Read
JSON support	Native JSONB (indexable)	JSON type (less mature indexing)
Window functions	Full support	Full support (8.0+)
CTEs / recursive queries	Excellent	Supported (8.0+)
Full-text search	Built-in (tsvector)	FULLTEXT indexes
GIS	PostGIS extension	Basic spatial types
Replication	Streaming, logical	Binlog-based, Group Replication
Typical use	Analytics, complex queries, fintech	High-read OLTP, e-commerce, web apps
Cloud managed	RDS, Cloud SQL, Neon, Supabase	RDS, PlanetScale, Aurora MySQL

When to pick MySQL: Simple relational schema, massive read traffic, team already on MySQL/LAMP, PlanetScale/Vitess sharding path, or company standard is MySQL.

When to pick PostgreSQL: Complex queries, JSONB, GIS, advanced types, or you need stricter default isolation for financial workloads.

MySQL Schema Design (InnoDB)

```
-- Users table (MySQL 8.0+)
CREATE TABLE users (
  id          CHAR(36) PRIMARY KEY DEFAULT (UUID()),
  name        VARCHAR(100) NOT NULL,
  email       VARCHAR(255) NOT NULL,
  password_hash TEXT NOT NULL,
  role        ENUM('user', 'admin') NOT NULL DEFAULT 'user',
  created_at  DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  updated_at  DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6) ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY uk_users_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Orders table
CREATE TABLE orders (
  id          CHAR(36) PRIMARY KEY DEFAULT (UUID()),
  user_id     CHAR(36) NOT NULL,
  status      ENUM('pending', 'paid', 'shipped', 'delivered', 'cancelled') NOT NULL,
  total       DECIMAL(10, 2) NOT NULL CHECK (total >= 0),
  created_at  DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  CONSTRAINT fk_orders_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  INDEX idx_orders_user_id (user_id),
  INDEX idx_orders_status_created (status, created_at DESC)
) ENGINE=InnoDB;

-- Products
CREATE TABLE products (
  id          CHAR(36) PRIMARY KEY DEFAULT (UUID()),
  name        VARCHAR(200) NOT NULL,
  slug        VARCHAR(200) NOT NULL,
  price       DECIMAL(10, 2) NOT NULL,
  stock       INT NOT NULL DEFAULT 0,
  category_id CHAR(36),
  created_at  DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  UNIQUE KEY uk_products_slug (slug),
  INDEX idx_products_category (category_id)
) ENGINE=InnoDB;
```

MySQL-specific notes:

- Always use **utf8mb4** charset (full Unicode including emoji).
- **ENGINE=InnoDB** — never MyISAM for transactional apps (no FK, table locks).
- **DATETIME(6)** or **TIMESTAMP** — prefer **DATETIME** for application timestamps (no 2038 limit).
- **AUTO_INCREMENT BIGINT** is common for high-volume IDs instead of UUID (better index locality).

mysql2 – Node.js Driver

```
import mysql from "mysql2/promise";

const pool = mysql.createPool({
  host: process.env.MYSQL_HOST,
  user: process.env.MYSQL_USER,
  password: process.env.MYSQL_PASSWORD,
  database: process.env.MYSQL_DATABASE,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0,
});

// Parameterized query (prevents SQL injection)
const [rows] = await pool.execute<mysql.RowDataPacket[]>(
  "SELECT u.name, SUM(o.total) AS total_spent FROM users u " +
  "LEFT JOIN orders o ON o.user_id = u.id " +
  "WHERE u.id = ? GROUP BY u.id",
  [userId]
);

// Transaction
const conn = await pool.getConnection();
try {
  await conn.beginTransaction();
  await conn.execute("UPDATE products SET stock = stock - ? WHERE id = ?", [qty, p
  await conn.execute("INSERT INTO order_items (order_id, product_id, quantity) VAL
    [orderId, productId, qty]);
  await conn.commit();
} catch (err) {
  await conn.rollback();
  throw err;
} finally {
  conn.release();
}
```

Prisma with MySQL

Same Prisma models work — change datasource only:

```
datasource db {
  provider = "mysql"
  url      = env("DATABASE_URL")
}

// User model - use @db.VarChar, @db.Decimal for MySQL types
model User {
  id          String   @id @default(uuid()) @db.Char(36)
  email       String   @unique @db.VarChar(255)
  passwordHash String   @map("password_hash")
  role        Role     @default(USER)
  orders      Order[]
  createdAt   DateTime @default(now()) @map("created_at")
  updatedAt   DateTime @updatedAt @map("updated_at")

  @@map("users")
}
```

```
# DATABASE_URL format for MySQL
# mysql://USER:PASSWORD@HOST:3306/DATABASE

npx prisma migrate dev --name init
npx prisma generate
```

MySQL Indexing & EXPLAIN

```
-- Show query plan
EXPLAIN ANALYZE
SELECT u.name, o.total
FROM users u
INNER JOIN orders o ON o.user_id = u.id
WHERE o.status = 'paid';

-- Composite index (leftmost prefix rule applies)
CREATE INDEX idx_orders_user_status ON orders(user_id, status);

-- Covering index - includes SELECT columns to avoid table lookup
CREATE INDEX idx_orders_cover ON orders(user_id, status, total, created_at);
```

EXPLAIN column	Good sign	Bad sign
type	ref , range , const	ALL (full table scan)
key	Index name shown	NULL
rows	Low estimate	High (millions)
Extra	Using index	Using filesort , Using temporary

MySQL Isolation & Locking (InnoDB)

Level	Default?	Phantom reads?	Use case
READ UNCOMMITTED	No	Yes	Rarely used
READ COMMITTED	No	Yes	Oracle-style apps
REPEATABLE READ	Yes (InnoDB)	Prevented via MVCC + gap locks	Most web apps
SERIALIZABLE	No	No	Strictest, slowest

```

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
-- ... queries ...
COMMIT;

```

Gap locks: InnoDB locks gaps between index records to prevent phantom inserts — can cause deadlocks under high concurrency. Keep transactions short.

Local MySQL Setup

```
# Docker
docker run -d --name mysql-dev \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=app_db \
  -p 3306:3306 \
  mysql:8.0

# Or Homebrew (macOS)
brew install mysql
brew services start mysql
mysql -u root -p
```

Managed options: AWS RDS MySQL, PlanetScale (Vitess — serverless scaling), Railway, Aiven.

7. Prisma ORM

Prisma provides **type-safe** database access with auto-generated TypeScript types. Works with **PostgreSQL, MySQL, SQLite, and MongoDB** — same client API, swap **provider** in schema.

Schema Definition (PostgreSQL example)

```
// prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id          String    @id @default(uuid())
  name        String
  email       String    @unique
  passwordHash String    @map("password_hash")
  role        Role      @default(USER)
  orders      Order[]
  createdAt   DateTime @default(now()) @map("created_at")
  updatedAt   DateTime @updatedAt @map("updated_at")

  @@map("users")
}

model Order {
  id          String    @id @default(uuid())
  userId      String    @map("user_id")
  user        User       @relation(fields: [userId], references: [id], onDelete: Cascade)
  status      OrderStatus @default(PENDING)
  total       Decimal    @db.Decimal(10, 2)
  items       OrderItem[]
  createdAt   DateTime    @default(now()) @map("created_at")

  @@index([userId])
  @@index([status, createdAt(sort: Desc)])
  @@map("orders")
}

model OrderItem {
  id          String    @id @default(uuid())
  orderId     String    @map("order_id")
  order       Order     @relation(fields: [orderId], references: [id], onDelete: Cascade)
  productId   String    @map("product_id")
  product     Product   @relation(fields: [productId], references: [id])
  quantity    Int
  unitPrice   Decimal    @map("unit_price") @db.Decimal(10, 2)

  @@unique([orderId, productId])
  @@map("order_items")
}
```

```
}

model Product {
  id          String      @id @default(uuid())
  name        String
  slug        String      @unique
  price       Decimal     @db.Decimal(10, 2)
  stock       Int         @default(0)
  orderItems OrderItem[]

  @@map("products")
}

enum Role {
  USER
  ADMIN
}

enum OrderStatus {
  PENDING
  PAID
  SHIPPED
  DELIVERED
  CANCELLED
}
```

Prisma Client Usage

```
import { PrismaClient } from "@prisma/client";

const prisma = new PrismaClient();

// Create with nested relations
const order = await prisma.order.create({
  data: {
    userId: user.id,
    total: 99.99,
    items: {
      create: [
        { productId: product1.id, quantity: 2, unitPrice: 29.99 },
        { productId: product2.id, quantity: 1, unitPrice: 39.99 },
      ],
    },
    include: { items: { include: { product: true } } },
  },
});

// Find with relations
const users = await prisma.user.findMany({
  where: { role: "ADMIN" },
  select: { id: true, name: true, email: true, _count: { select: { orders: true } } },
  orderBy: { createdAt: "desc" },
  take: 10,
});

// Update
await prisma.product.update({
  where: { id: productId },
  data: { stock: { decrement: quantity } },
});

// Raw SQL when needed
const result = await prisma.$queryRaw`
  SELECT u.name, COUNT(o.id) as order_count, SUM(o.total) as total_spent
  FROM users u
  LEFT JOIN orders o ON o.user_id = u.id
  GROUP BY u.id
  ORDER BY total_spent DESC
  LIMIT 10
`;
```

Prisma Migrations

```
# Create migration after schema change
npx prisma migrate dev --name add_order_status

# Apply migrations in production
npx prisma migrate deploy

# Reset database (dev only)
npx prisma migrate reset

# Generate client after schema change
npx prisma generate

# Open database GUI
npx prisma studio
```

8. SQL Joins & Queries

Join Types

```
-- INNER JOIN - only matching rows
SELECT u.name, o.id AS order_id, o.total
FROM users u
INNER JOIN orders o ON o.user_id = u.id
WHERE o.status = 'paid';

-- LEFT JOIN - all users, even without orders
SELECT u.name, COUNT(o.id) AS order_count
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.id, u.name;

-- RIGHT JOIN - all orders, even orphaned (rare)
SELECT u.name, o.total
FROM users u
RIGHT JOIN orders o ON o.user_id = u.id;

-- FULL OUTER JOIN - all from both tables
SELECT u.name, o.total
FROM users u
FULL OUTER JOIN orders o ON o.user_id = u.id;
```

Join Visual Reference



Subqueries vs CTEs

```
-- Subquery (correlated)
SELECT u.name,
       (SELECT COUNT(*) FROM orders o WHERE o.user_id = u.id) AS order_count
FROM users u;

-- CTE (Common Table Expression) - more readable
WITH user_stats AS (
  SELECT
    user_id,
    COUNT(*) AS order_count,
    SUM(total) AS total_spent,
    MAX(created_at) AS last_order
  FROM orders
  WHERE status = 'delivered'
  GROUP BY user_id
)
SELECT u.name, us.order_count, us.total_spent, us.last_order
FROM users u
JOIN user_stats us ON us.user_id = u.id
WHERE us.total_spent > 1000
ORDER BY us.total_spent DESC;
```

Window Functions

```
-- Rank users by spending per month
SELECT
  u.name,
  o.total,
  o.created_at,
  ROW_NUMBER() OVER (PARTITION BY u.id ORDER BY o.created_at) AS order_number,
  SUM(o.total) OVER (PARTITION BY u.id) AS lifetime_spent,
  AVG(o.total) OVER (PARTITION BY DATE_TRUNC('month', o.created_at)) AS monthly_avg
FROM orders o
JOIN users u ON u.id = o.user_id
WHERE o.status = 'delivered';
```

9. Transactions

Why Transactions?

Ensure **atomicity** — either all operations succeed or all roll back.

Scenario	Without Transaction	With Transaction
Transfer \$100 A→B	A debited, B credit fails → money lost	Both or neither
Create order + decrement stock	Order created, stock not updated → oversell	Atomic
Delete user + all data	Partial delete → orphaned records	Clean delete

PostgreSQL Transactions

```
BEGIN;  
  
UPDATE accounts SET balance = balance - 100 WHERE id = 'account-a';  
UPDATE accounts SET balance = balance + 100 WHERE id = 'account-b';  
  
-- Check constraint  
DO $$  
BEGIN  
  IF (SELECT balance FROM accounts WHERE id = 'account-a') < 0 THEN  
    RAISE EXCEPTION 'Insufficient funds';  
  END IF;  
END $$;  
  
COMMIT;  
-- Or ROLLBACK; on error
```

Prisma Transactions

```
// Sequential operations in one transaction
const result = await prisma.$transaction(async (tx) => {
  const product = await tx.product.findUnique({ where: { id: productId } });
  if (!product || product.stock < quantity) {
    throw new Error("Insufficient stock");
  }

  const order = await tx.order.create({
    data: {
      userId,
      total: product.price.mul(quantity),
      items: { create: [{ productId, quantity, unitPrice: product.price }] },
    },
  });

  await tx.product.update({
    where: { id: productId },
    data: { stock: { decrement: quantity } },
  });

  return order;
});
```

MongoDB Transactions

```
const session = await mongoose.startSession();
session.startTransaction();

try {
  await Order.create([{ userId, items, total }], { session });
  await Product.updateOne(
    { _id: productId },
    { $inc: { stock: -quantity } },
    { session }
  );
  await session.commitTransaction();
} catch (error) {
  await session.abortTransaction();
  throw error;
} finally {
  session.endSession();
}
```

Isolation Levels (PostgreSQL)

Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	Possible	Possible	Possible
Read Committed (default)	No	Possible	Possible
Repeatable Read	No	No	Possible
Serializable	No	No	No

10. Indexing Strategies

Index Types

Index Type	MongoDB	PostgreSQL	Use Case
Single field	<code>{ email: 1 }</code>	<code>CREATE INDEX ON users(email)</code>	Equality lookups
Compound	<code>{ role: 1, createdAt: -1 }</code>	<code>CREATE INDEX ON orders(status, created_at DESC)</code>	Multi-field queries
Unique	<code>{ email: 1 }, { unique: true }</code>	<code>CREATE UNIQUE INDEX</code>	Prevent duplicates
Text	<code>{ name: "text" }</code>	GIN/GiST on <code>to_tsvector</code>	Full-text search
Partial	<code>{ status: 1 }, { partialFilterExpression: { status: "active" } }</code>	<code>CREATE INDEX ... WHERE status = 'active'</code>	Subset of rows
Geospatial	<code>2dsphere</code>	PostGIS	Location queries

Index Rules

Rule	Explanation
Index columns in WHERE, JOIN, ORDER BY	Queries benefit from indexes
Leftmost prefix (compound)	Index <code>(a, b, c)</code> helps queries on <code>a</code> , <code>a+b</code> , <code>a+b+c</code> – not <code>b</code> alone
Don't over-index	Each index slows writes
Use EXPLAIN	<code>EXPLAIN ANALYZE</code> in PostgreSQL; <code>.explain()</code> in MongoDB
Covering index	Index includes all queried columns – no table lookup

EXPLAIN Example (PostgreSQL)

```
EXPLAIN ANALYZE
SELECT * FROM orders WHERE user_id = 'abc-123' AND status = 'paid' ORDER BY create

-- Good: Index Scan using idx_orders_user_status
-- Bad: Seq Scan on orders (full table scan – add index!)
```

MongoDB Index Performance

```
// Check if query uses index
db.orders.find({ userId: ObjectId("..."), status: "paid" }).explain("executionStats")

// executionStats.totalDocsExamined should ≈ nReturned
// If totalDocsExamined >> nReturned → collection scan or inefficient index
```

11. Aggregation & Analytics

MongoDB Aggregation Pipeline

```
db.orders.aggregate([
  // Stage 1: Filter
  { $match: { status: "delivered", createdAt: { $gte: ISODate("2024-01-01") } } },

  // Stage 2: Join users
  { $lookup: {
    from: "users",
    localField: "userId",
    foreignField: "_id",
    as: "user"
  }},
  { $unwind: "$user" },

  // Stage 3: Group by user
  { $group: {
    _id: "$userId",
    userName: { $first: "$user.name" },
    orderCount: { $sum: 1 },
    totalSpent: { $sum: "$total" },
    avgOrderValue: { $avg: "$total" },
  }},

  // Stage 4: Filter groups
  { $match: { totalSpent: { $gte: 1000 } } },

  // Stage 5: Sort and limit
  { $sort: { totalSpent: -1 } },
  { $limit: 10 },

  // Stage 6: Project final shape
  { $project: {
    _id: 0,
    userId: "$_id",
    userName: 1,
    orderCount: 1,
    totalSpent: { $round: ["$totalSpent", 2] },
    avgOrderValue: { $round: ["$avgOrderValue", 2] },
  }},
]);
```

Aggregation Stage Reference

Stage	Purpose
<code>\$match</code>	Filter documents (like WHERE)
<code>\$group</code>	Group and aggregate (like GROUP BY)
<code>\$project</code>	Select/transform fields (like SELECT)
<code>\$sort</code>	Sort results
<code>\$limit</code> / <code>\$skip</code>	Pagination
<code>\$lookup</code>	Join collections (like JOIN)
<code>\$unwind</code>	Deconstruct array field
<code>\$addFields</code>	Add computed fields
<code>\$facet</code>	Multiple pipelines in parallel
<code>\$bucket</code>	Group into ranges (histogram)

PostgreSQL Analytics Query

```
SELECT
  DATE_TRUNC('month', o.created_at) AS month,
  COUNT(DISTINCT o.user_id) AS unique_customers,
  COUNT(o.id) AS order_count,
  SUM(o.total) AS revenue,
  AVG(o.total) AS avg_order_value,
  PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY o.total) AS median_order
FROM orders o
WHERE o.status = 'delivered'
  AND o.created_at >= NOW() - INTERVAL '12 months'
GROUP BY DATE_TRUNC('month', o.created_at)
ORDER BY month;
```

12. Redis Caching Patterns

Redis Data Structures

Structure	Commands	Use Case
String	SET , GET , INCR	Cache, counters, rate limits
Hash	HSET , HGET , HGETALL	Object cache (user session)
List	LPUSH , RPOP , LRange	Queues, feeds, recent items
Set	SADD , SMEMBERS , SINTER	Tags, unique visitors
Sorted Set	ZADD , ZRange , ZRANK	Leaderboards, priority queues
Pub/Sub	PUBLISH , SUBSCRIBE	Real-time notifications
Stream	XADD , XREAD	Event logs, message queues

Cache-Aside Pattern (Most Common)

```
import Redis from "ioredis";

const redis = new Redis(process.env.REDIS_URL!);
const CACHE_TTL = 300; // 5 minutes

async function getUser(id: string) {
  const cacheKey = `user:${id}`;

  // 1. Check cache
  const cached = await redis.get(cacheKey);
  if (cached) return JSON.parse(cached);

  // 2. Cache miss – fetch from DB
  const user = await prisma.user.findUnique({ where: { id } });
  if (!user) return null;

  // 3. Store in cache
  await redis.setex(cacheKey, CACHE_TTL, JSON.stringify(user));

  return user;
}

async function updateUser(id: string, data: UpdateUserInput) {
  const user = await prisma.user.update({ where: { id }, data });

  // Invalidate cache on write
  await redis.del(`user:${id}`);

  return user;
}
```

Caching Patterns Comparison

Pattern	Read	Write	Consistency
Cache-Aside	App manages cache	Invalidate on write	Eventual
Read-Through	Cache fetches on miss	Cache handles	Eventual
Write-Through	Cache first	Sync to DB + cache	Strong
Write-Behind	Cache first	Async DB write	Eventual

Session Storage

```
import session from "express-session";
import RedisStore from "connect-redis";

app.use(session({
  store: new RedisStore({ client: redis }),
  secret: process.env.SESSION_SECRET!,
  resave: false,
  saveUninitialized: false,
  cookie: {
    secure: process.env.NODE_ENV === "production",
    httpOnly: true,
    maxAge: 24 * 60 * 60 * 1000, // 24 hours
    sameSite: "lax",
  },
}));
```

Rate Limiting with Redis

```
async function rateLimit(key: string, limit: number, windowSec: number): Promise<boolean> {
  const current = await redis.incr(key);
  if (current === 1) {
    await redis.expire(key, windowSec);
  }
  return current <= limit;
}

// Usage in middleware
const allowed = await rateLimit(`rl:${req.ip}`, 100, 900);
if (!allowed) return res.status(429).json({ error: "Rate limit exceeded" });
```

Leaderboard with Sorted Sets

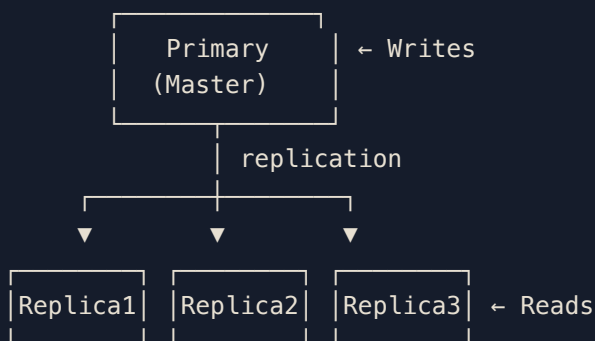
```
// Add score
await redis.zadd("leaderboard:2024", score, userId);

// Top 10
const top10 = await redis.zrevrange("leaderboard:2024", 0, 9, "WITHSCORES");

// User rank
const rank = await redis.zrevrank("leaderboard:2024", userId);
```

13. Database Architecture Patterns

Read Replicas



Connection Pooling

```
// Prisma handles pooling automatically
// For raw pg:
import pg from "pg";

const pool = new pg.Pool({
  connectionString: process.env.DATABASE_URL,
  max: 20, // max connections
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 2000,
});
```

Database Selection Flowchart

```
Need sub-ms latency?  
├ YES → Redis  
└ NO → Need complex joins/reporting?  
    ├── YES → PostgreSQL  
    └ NO → Schema changes frequently?  
        ├── YES → MongoDB  
        └ NO → Need ACID transactions?  
            ├── YES → PostgreSQL  
            └ NO → MongoDB (with Redis cache)
```

14. Interview Q&A

Q1: PostgreSQL vs MongoDB – when do you use each?

Answer: PostgreSQL for relational data needing JOINS, ACID transactions, and complex reporting — orders, payments, user accounts. MongoDB for flexible schemas, nested documents, high write throughput — product catalogs, activity logs, content management. Many apps use PostgreSQL as primary store and MongoDB for specific workloads, with Redis for caching.

Q2: Explain the CAP theorem.

Answer: In distributed systems, you can guarantee at most two of Consistency, Availability, and Partition tolerance. Since network partitions are inevitable, the real choice is CP (consistent but may reject requests during partition — MongoDB) or AP (available but may return stale data — Cassandra, Redis cluster). Single-node PostgreSQL is effectively CA.

Q3: Embedding vs referencing in MongoDB?

Answer: Embed when data is accessed together, bounded in size, and has a one-to-few relationship (user address, order line items). Reference when data grows unbounded, is shared across documents, or needs independent querying (user → orders, post → comments). Rule of thumb: embed what you query together; reference what grows independently.

Q4: How do you optimize slow database queries?

Answer: (1) Run EXPLAIN/EXPLAIN ANALYZE to identify full table scans. (2) Add indexes on WHERE, JOIN, ORDER BY columns. (3) Use covering indexes to avoid table lookups. (4) Denormalize for read-heavy patterns. (5) Add Redis cache for frequently read data. (6) Use connection pooling. (7) Paginate results — never return unbounded datasets.

Q5: Explain database transactions and isolation levels.

Answer: Transactions group operations into atomic units — all succeed or all roll back. Isolation levels control concurrent access: Read Committed (default, sees committed data), Repeatable Read (consistent reads within transaction), Serializable (full isolation, slowest). Use transactions for financial operations, inventory management, and any multi-step write that must be consistent.

Q6: What caching strategies do you use?

Answer: Cache-aside is most common — app checks Redis first, falls back to DB, stores result with TTL. Invalidate cache on writes. Use write-through for strong consistency needs. Cache hot data: user profiles, product listings, API responses. Set appropriate TTLs — short for dynamic data, long for static. Monitor cache hit rate; aim for >90% on cached endpoints.

Q7: How does Prisma differ from raw SQL or Mongoose?

Answer: Prisma generates type-safe TypeScript client from schema, supports PostgreSQL/MySQL/SQLite/MongoDB, handles migrations, and provides relation queries. Unlike raw SQL, you get compile-time type checking. Unlike Mongoose, Prisma supports SQL databases natively with JOINS. Trade-off: complex queries may need `$queryRaw`; less flexible than raw SQL for advanced analytics.

Q8: How do you design indexes?

Answer: Index columns used in WHERE, JOIN, and ORDER BY clauses. Compound indexes follow leftmost prefix rule. Use partial indexes for filtered subsets. Unique indexes enforce

constraints. Avoid indexing low-cardinality columns alone. Monitor with EXPLAIN — target index scans over sequential scans. Each index adds write overhead, so don't over-index.

Q9: PostgreSQL vs MySQL – when do you pick each?

Answer: Both are ACID SQL databases with InnoDB/PostgreSQL supporting transactions and foreign keys. Choose **PostgreSQL** for complex analytics, JSONB, GIS (PostGIS), advanced types, and when you need Read Committed as default with strong extension ecosystem. Choose **MySQL** for high-read OLTP web workloads, e-commerce, teams on LAMP/MEAN stacks, PlanetScale/Vitess sharding, or when the company standard is MySQL (common at Flipkart, Uber, Indian product companies). In MERN interviews, demonstrate you can work with **either** — Prisma abstracts most differences; know isolation defaults (PG: Read Committed, MySQL InnoDB: Repeatable Read) and charset (`utf8mb4`).

15. Practice Links & Resources

MongoDB

- [MongoDB University](#) — Free courses (M001, M121, M201)
- [MongoDB Docs](#) — Official reference
- [Mongoose Docs](#) — ODM guide

PostgreSQL

- [PostgreSQL Tutorial](#) — Step-by-step SQL tutorials
- [PostgreSQL Docs](#) — Official reference
- [Use The Index, Luke](#) — Indexing guide

MySQL

- [MySQL 8.0 Reference](#) — Official docs
- [MySQL Tutorial](#) — Beginner to advanced SQL
- [PlanetScale Docs](#) — Serverless MySQL (Vitess)
- [mysql2 npm](#) — Node.js driver

Redis

- [Upstash Docs](#) — Serverless Redis
- [Redis University](#) — Free Redis courses
- [Redis Commands Reference](#) — All commands

Prisma

- [Prisma Docs](#) — ORM reference
- [Prisma Data Guide](#) — Database concepts

Practice Projects

Project	Databases Used	Skills
E-commerce backend	PostgreSQL or MySQL + Redis	Schema design, transactions, caching
Blog platform	MongoDB + Redis	Document modeling, aggregation
High-traffic web app	MySQL + Redis	Read scaling, connection pooling
Analytics dashboard	PostgreSQL	Window functions, CTEs, indexing
Real-time leaderboard	Redis	Sorted sets, pub/sub
Multi-tenant SaaS	PostgreSQL + MongoDB	Polyglot persistence

Tools

Tool	Purpose
pgAdmin / DBeaver	PostgreSQL / MySQL GUI
MySQL Workbench	MySQL GUI
MongoDB Compass	MongoDB GUI
Redis Insight	Redis GUI
Prisma Studio	Prisma database browser
Atlas	Managed MongoDB
Supabase / Neon	Managed PostgreSQL
PlanetScale / RDS MySQL	Managed MySQL
Upstash	Serverless Redis

Databases Interview Q&A Bank – 50 Questions

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

50 interview questions with full detailed answers for MongoDB, PostgreSQL, MySQL, Redis, Structured Query Language (SQL), transactions, and indexing.

Target companies: Google, Amazon, Flipkart, Uber, Microsoft, Meta.

How to use: Pick 2 questions daily. Read the question, answer aloud in 2–3 minutes, then check the full answer.

Q1: What is the difference between SQL and NoSQL databases?

Answer: Structured Query Language (SQL) databases (PostgreSQL, MySQL) store data in fixed tables with rows and columns and enforce schema at write time. They excel at relationships using JOINS (combining tables) and full ACID (Atomicity, Consistency, Isolation, Durability) transactions. NoSQL (Not Only SQL) databases like MongoDB store flexible documents (often JSON-like Binary JSON called BSON) and scale horizontally more easily. SQL fits orders, payments, and reporting; NoSQL fits catalogs, logs, and rapidly changing schemas. Product companies often use both: one SQL database as the source of truth plus MongoDB or Redis for specific workloads.

Q2: Explain ACID properties with a money transfer example.

Answer: Atomicity means all steps succeed or none do — debiting one account and crediting another must be one unit; a crash mid-transfer rolls back. Consistency means rules (like balance ≥ 0) always hold. Isolation means concurrent transfers do not see each other's half-finished work. Durability means once committed, data survives power loss (written to disk or

replicated). Banks and e-commerce ledgers rely on ACID; interviewers at Flipkart or Amazon expect you to tie each letter to a real failure scenario.

Q3: What is the CAP theorem and how does it apply to MongoDB vs Redis?

Answer: CAP (Consistency, Availability, Partition tolerance) states that during a network partition you can prioritize Consistency or Availability, not both. Partition tolerance is required in distributed systems because networks fail. MongoDB replica sets default to CP — they may refuse writes if a majority of nodes is unreachable to avoid stale data. Redis Cluster is often AP for reads — clients may get slightly stale values but the system stays up. Single-node PostgreSQL on one server is effectively CA until you add replication.

Q4: When would you choose PostgreSQL over MySQL?

Answer: Choose PostgreSQL when you need advanced analytics (Common Table Expressions, window functions), rich types (JSONB, arrays, geographic information system extensions), strict standards compliance, or complex constraints. PostgreSQL handles concurrent writes and analytical queries well. Choose MySQL when the team standard is MySQL (common at Uber and Flipkart), you need simple high-read Online Transaction Processing (OLTP), or managed MySQL with Vitess sharding. Both are production-grade; know either for interviews.

Q5: When would you choose MongoDB over a SQL database?

Answer: MongoDB fits when schema evolves frequently (startup product iterations), documents are read together (user profile with nested preferences), write volume is high and relationships are simple, or you need flexible nested arrays without heavy JOINS. Avoid MongoDB as the only store when you need complex multi-table reports, strict relational integrity across many entities, or heavy ad-hoc analytics with window functions — PostgreSQL wins there.

Q6: What is BSON and how does it differ from JSON?

Answer: BSON (Binary JSON) is MongoDB's binary encoding of JSON-like documents. It supports additional types JSON lacks: Date, ObjectId (unique document identifier), Decimal128, and Binary data. BSON is faster to parse than text JSON and includes length prefixes for efficient traversal. In interviews, mention that drivers convert between JavaScript objects and BSON automatically — you still think in JSON shape at the application layer.

Q7: Explain embedding vs referencing in MongoDB schema design.

Answer: Embedding stores related data inside one document (order with line items array). Use when data is always fetched together, size is bounded, and relationship is one-to-few. Referencing stores an identifier pointing to another collection (user document with order IDs or order with userId). Use when related data grows unbounded, is shared across documents, or needs independent queries. Over-embedding causes 16 MB document limit issues; over-referencing causes application-level JOINS similar to SQL.

Q8: How do MongoDB indexes work and what is the default `_id` index?

Answer: MongoDB uses B-tree indexes (Balanced tree for range and equality lookups). Every collection automatically has a unique index on `_id`. Queries without a supporting index perform a collection scan (reading every document) — slow at scale. Create indexes on fields used in filter, sort, and lookup stages. Compound indexes follow the leftmost prefix rule: index on `{a:1,b:1}` helps queries on `a` alone or `a+b`, not `b` alone.

Q9: What is the MongoDB aggregation pipeline?

Answer: The aggregation pipeline processes documents through stages like `$match` (filter), `$group` (aggregate), `$lookup` (join another collection), `$sort`, and `$project` (reshape fields). It is MongoDB's answer to SQL GROUP BY and JOIN for analytics. Example: count orders per user with `$match` on date range, `$group` by userId, `$sort` by count. Use

\$explain on aggregation to verify index usage — interviewers ask this for dashboard and reporting workloads.

Q10: Explain MongoDB replica sets and failover.

Answer: A replica set is one primary node (accepts writes) and secondary nodes (replicate the oplog — operation log). If the primary dies, an election promotes a secondary. Clients should use a connection string with replica set name for automatic failover. Write concern **majority** ensures data is replicated before acknowledging — important for durability. Read concern **majority** avoids stale reads after failover. At Google-scale interviews, mention trade-off between durability and write latency.

Q11: What is MongoDB sharding?

Answer: Sharding splits data across multiple shards (each often a replica set) using a shard key. The mongos router directs queries to the correct shard. A bad shard key (low cardinality or monotonic like timestamp-only) creates hot shards — one server gets all writes. Good keys distribute evenly (hashed userId). Sharding is for horizontal scale when a single replica set exceeds disk, memory, or write throughput limits.

Q12: What are the main SQL JOIN types?

Answer: INNER JOIN returns rows where both tables match the join condition. LEFT (OUTER) JOIN keeps all rows from the left table and null-fills missing right matches — common for optional relations. RIGHT JOIN is the mirror. FULL OUTER JOIN keeps unmatched rows from both sides (PostgreSQL supports it; MySQL does not natively). CROSS JOIN is the Cartesian product — every row paired with every row. Interviews often ask you to write a query finding users with no orders — that is LEFT JOIN ... WHERE order.id IS NULL.

Q13: Explain database normalization (1NF, 2NF, 3NF) in plain language.

Answer: First Normal Form (1NF): atomic columns, no repeating groups — no storing multiple phone numbers in one comma-separated field. Second Normal Form (2NF): no partial dependency on a composite key — every non-key column depends on the whole primary key. Third Normal Form (3NF): no transitive dependency — columns depend only on the primary key, not on other non-key columns. Normalization reduces redundancy and update anomalies. Denormalization is intentional duplication for read speed — know both.

Q14: When is denormalization acceptable?

Answer: Denormalize when read performance dominates, joins are expensive at scale, and data is read far more than updated. Examples: storing `authorName` on each blog post to avoid joining users on every feed load, or embedding product snapshot on order line items so historical price stays correct. Trade-offs: larger storage, risk of stale copied fields, harder updates. Cache (Redis) is another form of controlled denormalization.

Q15: What is a B-tree index and why do databases use it?

Answer: A B-tree is a balanced tree where each node holds many keys and children, keeping tree height low. It supports $O(\log n)$ lookups, range scans (`WHERE age BETWEEN 20 AND 30`), and ordered retrieval for ORDER BY. Most relational databases and MongoDB default to B-tree variants. Interview tip: equality and range queries on indexed columns avoid full table scans; string prefix searches can use indexes, but `LIKE '%suffix'` often cannot.

Q16: What is a compound index and the leftmost prefix rule?

Answer: A compound index spans multiple columns, e.g. `(country, city, zip)`. The leftmost prefix rule means the index helps queries filtering on `country`, or `country+city`, or all three — but not `city` alone without `country`. Order matters: put high-selectivity or equality columns first, then range columns. Wrong column order wastes the index — a common senior interview question at Amazon.

Q17: What is a covering index?

Answer: A covering index includes all columns the query needs, so the database reads only the index without touching the main table (index-only scan). Example: index on `(user_id, created_at)` for `SELECT created_at FROM orders WHERE user_id = ?` — if `created_at` is in the index, no heap lookup. Smaller result sets and less input/output make queries much faster. In PostgreSQL, use `INCLUDE` columns for this pattern.

Q18: What is a partial index?

Answer: A partial index indexes only rows matching a condition, e.g. `WHERE deleted_at IS NULL`. Smaller index size, faster writes, faster lookups for the common case (active users only). PostgreSQL and MongoDB support partial indexes. Use when queries always filter the same subset — soft-deleted records, pending orders, or published posts only.

Q19: How do you use EXPLAIN to optimize a slow query?

Answer: Run `EXPLAIN` or `EXPLAIN ANALYZE` (PostgreSQL/MySQL) to see the execution plan: sequential scan vs index scan, estimated rows, join order, and actual timings. Look for high cost nodes, nested loop joins on large tables without indexes, and sort operations on huge datasets. Fix by adding indexes, rewriting JOINS, limiting selected columns, or paginating. At Microsoft interviews, walk through one real slow query you fixed.

Q20: What is the N+1 query problem?

Answer: N+1 happens when code loads N parent records (1 query) then runs one query per parent for related data (N queries) — e.g. load 100 posts, then 100 queries for each author's name. Total: 101 queries. Fix with JOIN, batch `WHERE author_id IN (...)`, ORM eager loading (Prisma `include`, Mongoose `populate`), or DataLoader pattern in GraphQL. This bug appears constantly in Node.js backends and is a favorite Flipkart question.

Q21: What is connection pooling and why does it matter?

Answer: Opening a database connection is expensive (TCP handshake, authentication, memory on server). A pool maintains reusable connections; the app borrows and returns them. Without pooling, traffic spikes exhaust connections or add latency. Configure max pool size based on database limits — PostgreSQL default `max_connections` is often 100. Prisma and `pg` Pool handle this; watch for pool exhaustion under load tests.

Q22: Explain transaction isolation levels.

Answer: Isolation levels control what concurrent transactions see. Read Uncommitted: dirty reads possible (rarely used). Read Committed: only committed data visible — PostgreSQL default. Repeatable Read: same row reads consistent within transaction — MySQL InnoDB default. Serializable: full isolation, may use locks and reduce concurrency. Higher isolation prevents anomalies (dirty read, non-repeatable read, phantom read) but increases lock contention. Pick the lowest level that keeps data correct.

Q23: What is a database deadlock and how do you prevent it?

Answer: Deadlock occurs when transaction A locks row 1 waiting for row 2 while transaction B locks row 2 waiting for row 1 — neither can proceed. Databases detect and abort one transaction. Prevention: acquire locks in consistent order (always lock user then order), keep transactions short, use row-level locks wisely, avoid unnecessary `SELECT FOR UPDATE`. Retry aborted transactions with exponential backoff in application code.

Q24: Compare optimistic vs pessimistic locking.

Answer: Pessimistic locking locks rows upfront (`SELECT FOR UPDATE`) — safe for high contention inventory (last ticket in concert). Optimistic locking uses a version column — read row with version 5, update only if still version 5; if someone else updated, retry. Optimistic fits low collision e-commerce carts; pessimistic fits financial debits. MongoDB uses atomic operators like `findOneAndUpdate` with conditions for optimistic patterns.

Q25: What are read replicas and what is replication lag?

Answer: Read replicas copy data from the primary for read scaling. Writes go to primary; reads can go to replicas. Replication lag is delay before a write appears on replicas — milliseconds to seconds under load. Risk: user updates profile, immediately reads replica, sees old data. Mitigations: read-your-writes routing (same user to primary briefly), session stickiness, or accept eventual consistency for feeds. Uber and Netflix use replicas heavily.

Q26: Explain database sharding vs partitioning.

Answer: Partitioning splits data within one database server (PostgreSQL table partitions by date range on one instance). Sharding splits across multiple servers (user IDs 1–1M on shard A, 1M+ on shard B). Partitioning simplifies archival (drop old partition); sharding scales beyond one machine's limits. Both require careful key choice to avoid hot spots. Vitess shards MySQL; MongoDB has native sharding.

Q27: What Redis data structures should you know for interviews?

Answer: Strings (cache values, counters with INCR), Lists (queues with LPUSH/BRPOP), Sets (unique tags, mutual followers), Sorted Sets (leaderboards with score-based rank), Hashes (object fields like user session), Streams (event logs), HyperLogLog (approximate unique counts), Bitmap (daily active users). Redis is in-memory — sub-millisecond latency. Know which structure maps to which product feature.

Q28: Explain the cache-aside pattern.

Answer: Application checks Redis first. On miss, read database, store in Redis with Time To Live (TTL), return to client. On write, update database then delete or update cache (cache invalidation). Most common caching pattern at product companies. Set TTL to balance freshness vs load — product catalog might be 5 minutes; user session might be 24 hours. Monitor hit rate; below 80% on hot keys suggests wrong keys or TTL.

Q29: What is write-through vs write-behind caching?

Answer: Write-through: every write updates cache and database synchronously — strong consistency, higher write latency. Write-behind (write-back): write to cache first, async flush to database — faster writes, risk of data loss on crash. Read-through: cache layer loads from DB on miss automatically (client always talks to cache). Interview answer: use cache-aside for most REST APIs; write-through when stale cache is unacceptable.

Q30: How does Redis pub/sub work and when would you use it?

Answer: Publishers send messages to channels; subscribers receive in real time. Fire-and-forget — no persistence if subscriber offline. Use for live notifications, chat fan-out, cache invalidation signals across app servers. For durable messaging use Redis Streams, Kafka, or RabbitMQ. At Meta-scale design questions, pub/sub is for ephemeral broadcast, not order processing.

Q31: What are Redis sorted sets used for?

Answer: Sorted sets store unique members each with a numeric score, kept sorted. Commands: ZADD (add with score), ZRANK/ZREVRANK (position), ZRANGE (top N). Perfect for leaderboards, rate limiter sliding windows, priority task queues. Flipkart-style gamification and Uber driver dispatch scoring often map to sorted sets or similar rank structures.

Q32: Explain Redis TTL and eviction policies.

Answer: TTL (Time To Live) expires keys automatically — `SET key value EX 3600`. When memory is full, eviction policies apply: `allkeys-lru` evicts least recently used keys globally; `volatile-lru` evicts only keys with TTL. Choose policy based on whether all keys are cache (allkeys) or mix of cache and required data. Monitor memory usage — Redis is fast until Out Of Memory (OOM) kills writes.

Q33: Compare Redis RDB snapshots vs AOF persistence.

Answer: RDB: periodic point-in-time snapshots — compact, fast restart, may lose last few minutes of writes. AOF (Append Only File): logs every write — more durable, larger files, slower restart (can rewrite). Many teams use both or accept Redis as pure cache with rebuild-from-DB on restart. For session store durability, enable AOF with `everysec` fsync balance.

Q34: What is Prisma and how does it compare to raw SQL?

Answer: Prisma is a TypeScript Object-Relational Mapper (ORM) generating a type-safe client from a schema file. It supports PostgreSQL, MySQL, SQLite, MongoDB. Benefits: autocomplete, compile-time errors, migrations. Raw SQL via `$queryRaw` for complex reports. Compared to raw SQL: less boilerplate, easier refactors; trade-off is abstraction leakage on very complex queries. Standard in modern MERN stacks.

Q35: What is Mongoose and what problems does it solve?

Answer: Mongoose is the Object Document Mapper (ODM) for MongoDB in Node.js. It adds schema validation, defaults, middleware hooks (pre-save hashing passwords), virtuals, and population (join-like loading of references). Without Mongoose, you validate in application code only. Know `Schema`, `Model`, indexes in schema, and that strict mode strips unknown fields by default.

Q36: What are database migrations and best practices?

Answer: Migrations are versioned schema changes (add column, create index) applied consistently across environments. Tools: Prisma Migrate, Flyway, Liquibase, Alembic. Best practices: never edit production manually; backward-compatible steps (add nullable column before backfill); deploy code that works with old and new schema; create indexes `CONCURRENTLY` on PostgreSQL to avoid locking. Failed migration rollback plan is a senior interview topic.

Q37: How would you schema-design an e-commerce order system?

Answer: Users table (id, email). Products (id, sku, price — price snapshot on order line, not live product price). Orders (id, user_id, status, total). Order_items (order_id, product_id, quantity, unit_price at purchase). Payments separate with transaction id. Use foreign keys in SQL for integrity. Index `(user_id, created_at)` for order history. Redis cache product catalog; SQL for orders and payments with transactions.

Q38: What is polyglot persistence?

Answer: Using multiple database types in one system for the right job: PostgreSQL for transactions, MongoDB for product catalog, Redis for sessions and cache, Elasticsearch for search. MERN apps often say MongoDB only, but Flipkart/Uber-scale systems mix stores. Interview: justify each choice with access pattern, consistency needs, and query shape — not buzzwords.

Q39: What is eventual consistency and when is it acceptable?

Answer: Eventual consistency means replicas converge over time; reads may temporarily return old values. Acceptable for social feeds, view counts, recommendation scores, non-critical profile fields. Not acceptable for account balance, inventory deduction, or idempotent payment id without strong checks. Always state the business impact of stale data in interviews.

Q40: What is a primary key vs a unique index?

Answer: Primary key uniquely identifies a row; only one per table; often clustered (physical row order follows it in MySQL InnoDB). Unique index enforces uniqueness on column(s) but allows multiple unique indexes and one NULL in SQL (unless NOT NULL). Use surrogate keys (auto-increment id, UUID) vs natural keys (email) — UUIDs ease distributed inserts; integers are smaller and faster to join.

Q41: What are foreign keys and should you always use them?

Answer: Foreign keys enforce referential integrity — cannot insert order with invalid user_id. Cascades can DELETE user and related rows automatically. Benefits: database-enforced consistency. Drawbacks: complicates sharding, migrations, and high-write distributed systems — some teams enforce in application layer instead. For most MERN/SQL apps on one database, use foreign keys; at shard scale, discuss trade-offs openly.

Q42: What are SQL window functions? Give an example.

Answer: Window functions compute across related rows without collapsing groups like GROUP BY. Example: `ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC)` ranks employees within each department. Others: `LAG`, `LEAD`, running totals. PostgreSQL excels here. Use for top-N per group, moving averages, and analytics dashboards — common Amazon data interview SQL question.

Q43: What are prepared statements and why use them?

Answer: Prepared statements parse SQL once, execute many times with different parameters — faster and immune to SQL injection when used correctly (`$1` placeholders, never string concat). ORMs use them by default. Example attack without preparation: input `' OR '1'='1` breaks naive queries. Always parameterize user input — security and performance win.

Q44: How does MySQL InnoDB differ from MyISAM?

Answer: InnoDB supports transactions, row-level locking, foreign keys, and crash recovery — default and required for production. MyISAM has table-level locks, no transactions — legacy, avoid. InnoDB stores rows in primary key order (clustered index). Interviewers at Indian product companies expect you to say InnoDB for any transactional workload.

Q45: What is PostgreSQL JSONB and when to use it?

Answer: JSONB stores JSON in binary parsed form — indexable with GIN indexes, queryable with operators (`->` , `->>` , `@>`). Use for semi-structured attributes (user preferences, metadata) that change often without migrations. Not a replacement for normalized relations — avoid storing entire order graph in one JSON blob if you need relational queries. Hybrid model is common at startups scaling to Microsoft-level interviews.

Q46: Explain the difference between DELETE, TRUNCATE, and DROP.

Answer: DELETE removes rows matching condition, row-by-row, can rollback in transaction, triggers fire. TRUNCATE removes all rows quickly by deallocating pages — faster, usually cannot rollback, resets auto-increment. DROP removes entire table structure and data. Production: prefer soft delete (`deleted_at`) for audit trails; TRUNCATE staging tables; never DROP without backups.

Q47: How do you handle pagination at scale – OFFSET vs cursor?

Answer: OFFSET/LIMIT (`LIMIT 20 OFFSET 100000`) scans and skips 100k rows — slow on large tables. Cursor (keyset) pagination uses `WHERE id > last_seen_id ORDER BY id LIMIT 20` — constant time with index. Use cursor for infinite scroll feeds (Instagram, Flipkart product lists). OFFSET is fine for small admin pages.

Q48: How would you choose a database for a new startup MVP vs at scale?

Answer: MVP: one managed PostgreSQL or MongoDB on Railway/Supabase/Atlas — minimize ops. Add Redis when read latency hurts. At scale: read replicas, connection pooling (PgBouncer), cache hot paths, consider sharding when vertical scale (bigger server) fails. Match database to team skills and query patterns. Interviewers want phased evolution, not premature Kafka plus 5 databases on day one.

Q49: What Redis command would you use for a rate limiter?

Answer: Fixed window: INCR key, EXPIRE on first increment, reject if count > limit. Sliding window: sorted set ZADD with timestamp score, ZREMRANGEBYSCORE to drop old entries, ZCARD to count. Token bucket can be simulated with Lua script for atomicity. Return HTTP 429 Too Many Requests when exceeded — standard API design at Uber and public APIs.

Q50: Summarize how you would answer 'design the data layer for a ride-sharing app' in an interview.

Answer: Users and drivers in SQL with geo indexes or PostGIS. Trips table with status state machine, transactional payment records. Redis for driver location cache and nearby driver queries (GEORADIUS). MongoDB optional for trip event logs. Read replicas for trip history. Strong consistency on payment; eventual on driver location. Mention partition by city at scale. Connect each store to a concrete access pattern — that is what L5 interviews reward.